EE-170L Computer Programming Lab

Laboratory Report — No. 10

More About Functions in C++

Student Name	Zawar Shah
Registration No.	BF25NWELE0677
Course	EE-170L – Computer Programming Lab
Lab No.	10
Topic	Call by Value, Call by Reference, Default Parameters
Semester	Spring 2026
Institution	UET Peshawar

1. Lab Objectives

- Understand the difference between call by value and call by reference in C++.
- Recognize that call by value creates a local copy, leaving the original variable unchanged.
- Use the & operator in call by reference to allow a function to modify the caller's variable.
- Implement functions with default parameter values for flexible calling.
- Demonstrate how call by reference enables a single function to return multiple values.
- Apply both calling conventions to practical problems such as swapping and division.

2. Theoretical Background

2.1 Calling Function: Call By Value

In call by value, a copy of the argument is created and passed to the function. Any changes made inside the function affect only the local copy — the original variable in the caller remains unchanged. This is the default behaviour in C++.

```
#include <iostream>
using namespace std;
void func(int);
int main()
{
    int i = 10;
    func(i);
    cout << "The value of i remains " << i << endl;
    return 0;
}
void func(int i)
{
    i = i + 10;
    cout << "Inside function, i changed to " << i << endl;
}
```

Output:

```
Inside function, i changed to 20
```

```
The value of i remains 10
```

2.2 Call By Value — Swap Example (Fails to Swap Originals)

Because swapping works only on local copies, the original variables x and y are unchanged after the function returns:

```
void swapping(int x1, int y1)
{
    int z1 = x1; x1 = y1; y1 = z1;
    cout << "Inside: x1=" << x1 << "  y1=" << y1 << endl;
}
```

Output:

```
Before: x = 50  y = 70
Inside: x1 = 70  y1 = 50
After:  x = 50  y = 70    <-- originals unchanged
```

2.3 Calling Function: Call By Reference

In call by reference, the & operator is used in the parameter list. The function receives the memory address of the argument and operates directly on the original variable. Changes persist after the function returns.

```
#include <iostream>
using namespace std;
void swapping(int &, int &);
int main()
{
    int x = 50, y = 70;
    cout << "Before: x=" << x << "  y=" << y << endl;
    swapping(x, y);
    cout << "After:  x=" << x << "  y=" << y << endl;
    return 0;
}
void swapping(int &x1, int &y1)
{
    int z1 = x1; x1 = y1; y1 = z1;
}
```

Output:

```
Before: x = 50  y = 70
After:  x = 70  y = 50    <-- originals are swapped
```

2.4 Default Values in Parameters

A function can specify default values for its trailing parameters. If the caller omits those arguments, the defaults are used. If the caller provides a value, the default is overridden:

```
#include <iostream>
using namespace std;
int divide(int a = 8, int b = 2)
{
    return a / b;
}
int main()
{
    cout << divide()      << endl;  // 8/2 = 4
    cout << divide(12)    << endl;  // 12/2 = 6
    cout << divide(63, 7) << endl;  // 63/7 = 9
    return 0;
}
```

Output:

```
4
6
9
```

3. Lab Tasks & Solutions

Task 1 — multiply() — Function with Parameters

Declare a void function 'multiply' that takes two integer parameters num1 and num2. Calculate the product inside the function and display it. Call from main() with two numbers.

```
#include <iostream>
using namespace std;
void multiply(int num1, int num2);
int main()
{
    multiply(6, 9);
    multiply(15, 4);
    return 0;
}
void multiply(int num1, int num2)
{
    int product = num1 * num2;
    cout << num1 << " x " << num2 << " = " << product << endl;
}
```

Output:

```
6 x 9 = 54
15 x 4 = 60
```

How it works:

- multiply() is a void function — it computes and prints but returns nothing.
- num1 and num2 are local copies of the arguments passed (call by value).
- The function can be called multiple times with different argument pairs.

Task 2 — getSquare() — Function with Return Value

Declare a function 'getSquare' that takes an integer 'number', calculates its square, and returns the result. Call from main() and display the returned value.

```
#include <iostream>
using namespace std;
int getSquare(int number);
int main()
{
    int n;
    cout << "Enter a number: ";
    cin >> n;
    int sq = getSquare(n);
    cout << n << " squared = " << sq << endl;
    return 0;
}
int getSquare(int number)
{
    return number * number;
}
```

Sample Output (input = 7):

```
Enter a number: 7
7 squared = 49
```

How it works:

- getSquare() returns an int — the square of the input number.
- The returned value is stored in sq in main() and then printed.
- This demonstrates the correct pattern for a function with a meaningful return value.

4. Call by Value vs Call by Reference — Key Differences

Feature	Call by Value	Call by Reference
What is passed	A copy of the variable	Memory address of the variable
Syntax	void f(int x)	void f(int &x)
Original changed	No — original is safe	Yes — function modifies original
Use when	Read-only; no side effects	Must modify caller's variable
Swap example	Fails — swaps local copies only	Works — swaps the originals
Multiple returns	Not directly possible	Yes — via multiple ref parameters

5. Conclusion

This lab deepened the understanding of C++ functions by covering three advanced topics. Key takeaways:

- **Call by Value:** Passes a copy — the original is protected from modification. Use for read-only operations.
- **Call by Reference:** Passes the address — the function can directly modify the original variable. Essential for swap and multi-output functions.
- **& Operator:** Adding & before a parameter in the prototype and definition turns it into a reference parameter.
- **Default Parameters:** Assign default values in the prototype. The caller may omit those arguments and the defaults apply automatically.
- **Choosing a Convention:** Use call by value for safety; use call by reference when the function must change the caller's data or return more than one value.

These calling conventions are essential for writing efficient, correct C++ programs and form the foundation for pointers and object-oriented programming in future labs.